

Depuración

Detectar y corregir errores en tiempo de ejecución

¿Qué es depurar?

Depurar consiste en ejecutar el programa de forma controlada, deteniéndolo donde tú decides, para observar y modificar su estado mientras está en marcha. La herramienta que lo hace posible es el depurador (debugger), integrado en IntelliJ.

✗ Sin depurador

- Añadir `System.out.println` por todo el código
- Recompilar y ejecutar cada vez que se añade un print
- Difícil saber exactamente dónde está el error
- El código de depuración queda mezclado con el real

✓ Con el depurador

- El programa se detiene en el punto exacto que eliges
- Ves el valor de todas las variables en ese instante
- Avanzas línea a línea para seguir la ejecución
- Puedes modificar variables en caliente sin recompilar

Un error que solo se ve en ejecución

Código con error de lógica

```
int[] numbers = {1, 2, 3, 4, 5};
int sum = 0;

for (int i = 0; i <= numbers.length; i++) { // ← ERROR: <= en vez de <
    sum += numbers[i];
}

System.out.println("Suma: " + sum);
```

Excepción en ejecución (el programa compila sin error)

```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException:
    Index 5 out of bounds for length 5
    at Main.main(Main.java:5)
```

El array solo tiene índices del 0 al 4. La condición `i <= numbers.length` permite que `i` llegue a 5. Con el depurador, paras en el bucle y ves que `i` vale 5 antes del fallo. Solución: cambiar `<=` por `<`.

Iniciar el depurador en IntelliJ

Hay tres formas equivalentes de lanzar el depurador en IntelliJ:

Clic en el margen

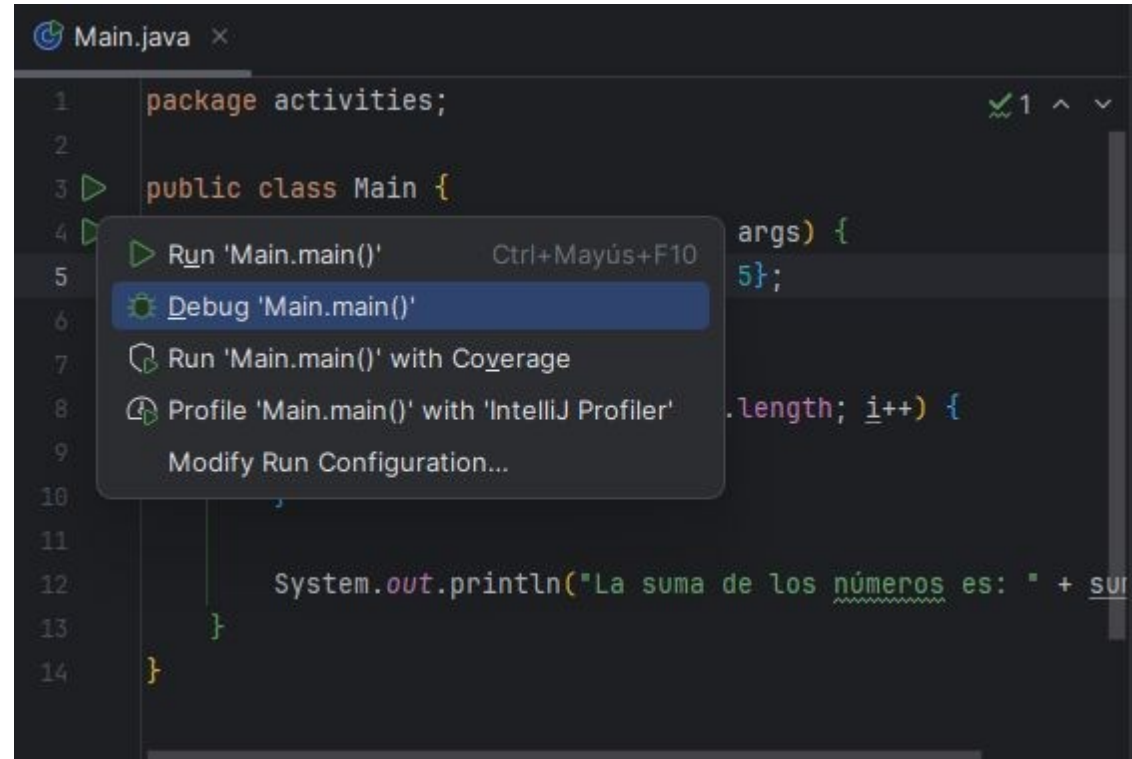
Zona gris junto al número de línea → Debug 'Main'

Menú Run

Run → Debug 'Main'

Shift + F9

Atajo de teclado directo



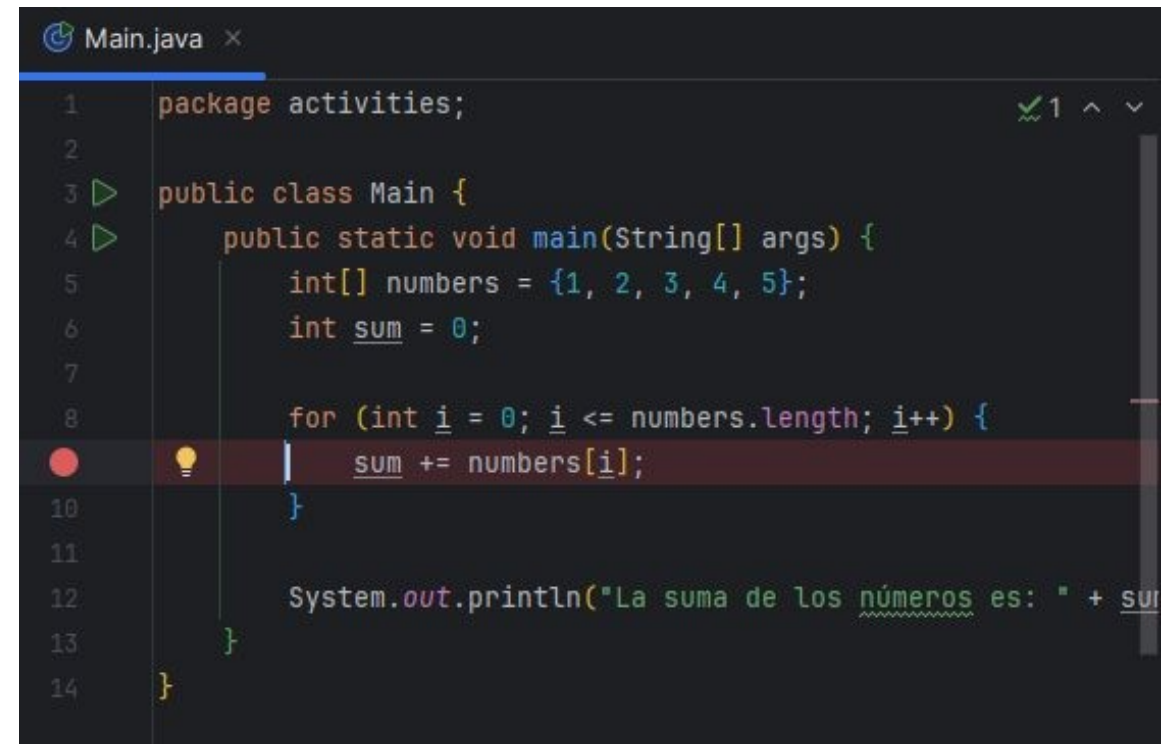
⚠ Sin breakpoints no pasa nada distinto: el programa se ejecuta de principio a fin igual que con Run normal. Para que la depuración tenga sentido, hace falta marcar al menos un punto donde el programa se pare.

Puntos de interrupción (Breakpoints)

Un breakpoint es una marca en una línea de código que le dice al depurador: detén aquí la ejecución. Cuando el programa llega a esa línea durante la depuración, se para y puedes inspeccionarlo.

Cómo añadir y quitar un breakpoint

- Haz clic en el margen izquierdo, junto al número de línea
- Aparece una bola roja y la línea se resalta
- Atajo: Ctrl + F8 (también añade y elimina)
- Para quitarlo: clic de nuevo sobre la bola roja



```
1 package activities;
2
3 public class Main {
4     public static void main(String[] args) {
5         int[] numbers = {1, 2, 3, 4, 5};
6         int sum = 0;
7
8         for (int i = 0; i <= numbers.length; i++) {
9             sum += numbers[i];
10        }
11
12        System.out.println("La suma de los números es: " + sum);
13    }
14 }
```

The screenshot shows an IDE window titled 'Main.java'. The code is a Java program that calculates the sum of an array of numbers. A breakpoint, represented by a red dot, is set on line 9, which is the line containing the statement `sum += numbers[i];`. The line is highlighted in a dark red color. To the left of the line number 9, there is a small yellow lightbulb icon, indicating a breakpoint. The line numbers 1 through 14 are visible on the left side of the code editor.

💡 Los breakpoints se guardan entre sesiones. Si cierras IntelliJ y lo abres de nuevo, los breakpoints que tenías siguen ahí.

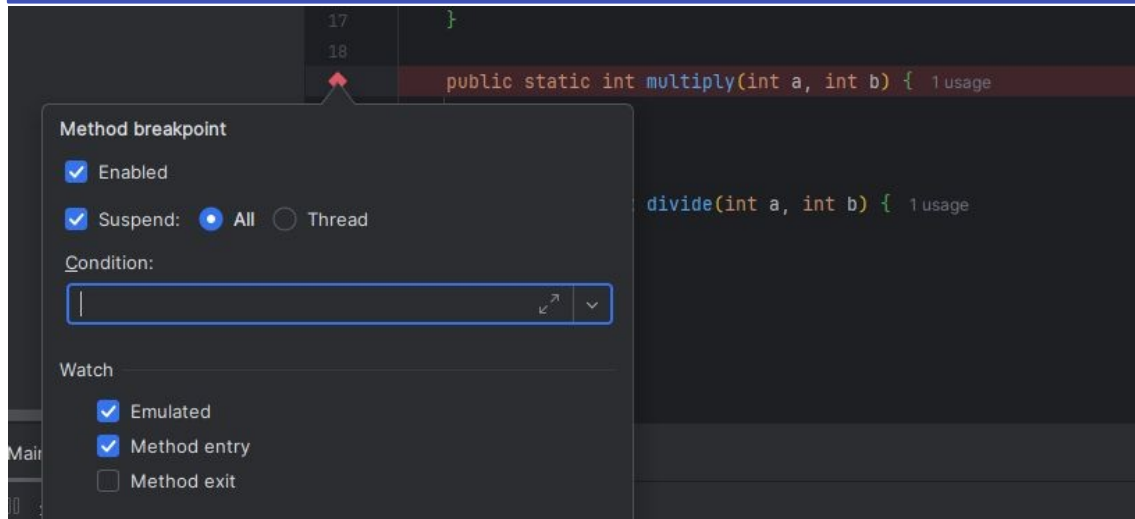
Tipos de breakpoints

IntelliJ distingue varios tipos de breakpoints según qué hace que se active la parada:

Tipo	Se activa cuando...	Cuándo usarlo
De línea	La ejecución llega a la línea de código marcada	El más habitual. Punto de partida en cualquier depuración.
De método	El programa entra o sale de un método concreto	Para comprobar condiciones de entrada/salida sin buscar la línea exacta.
Field Watchpoint	Se lee o modifica un atributo de instancia	Detectar qué parte del código está cambiando una variable de objeto sin que lo esperes.
De excepción	Se lanza una excepción concreta en cualquier punto	Encontrar el origen real de una excepción inesperada. Se configura desde Ctrl+Shift+F8.

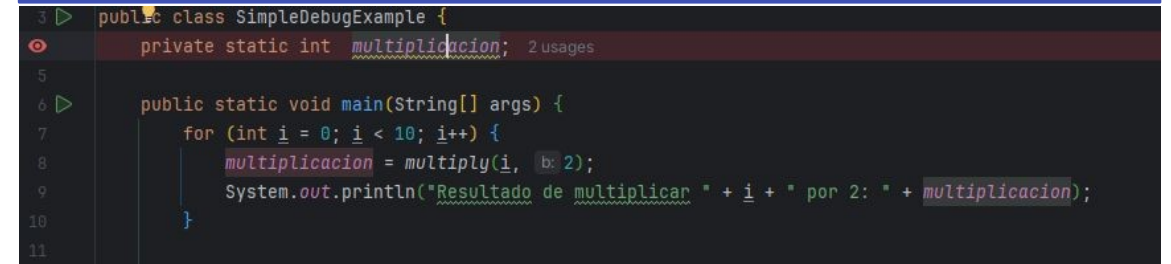
Tipos de breakpoints — Capturas

De método



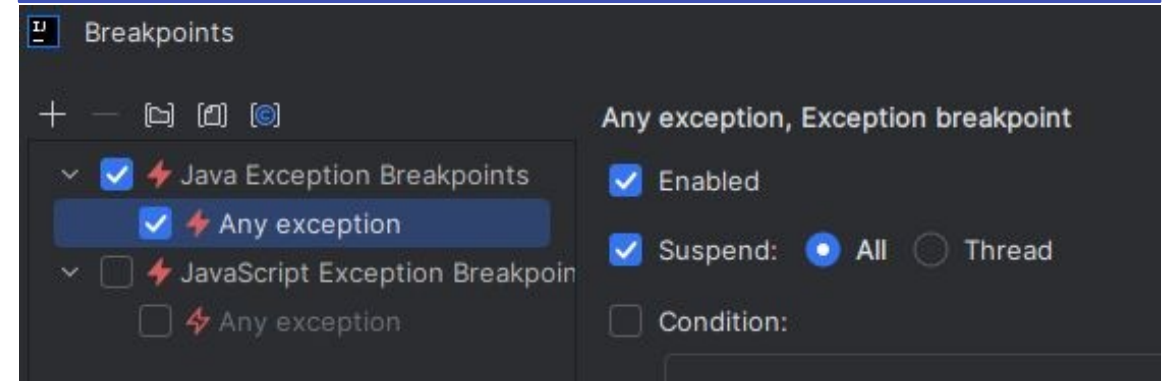
Sobre la firma del método. Permite elegir si se activa al entrar, al salir o en ambos.

Field Watchpoint



Sobre la declaración de un atributo. Se activa cuando se lee o escribe ese campo.

De excepción

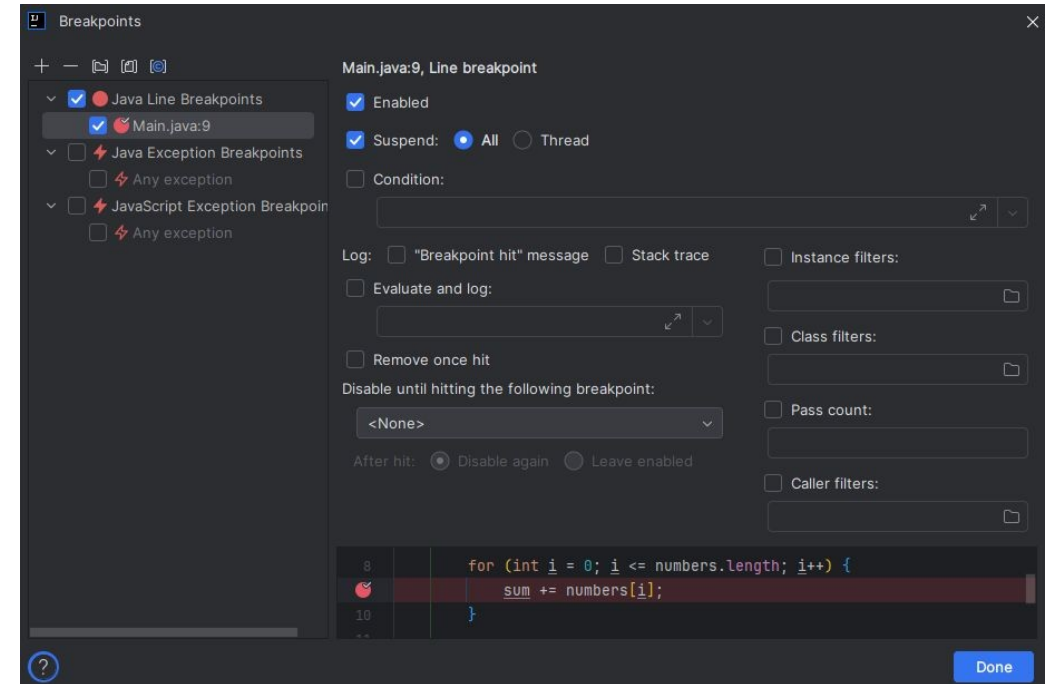


Desde View Breakpoints. Se activa al lanzarse la excepción, antes de que el catch la capture.

Configuraciones avanzadas de breakpoints

Desde View Breakpoints (Ctrl + Shift + F8) cada breakpoint admite configuraciones adicionales en lugar de limitarse a «parar siempre aquí»:

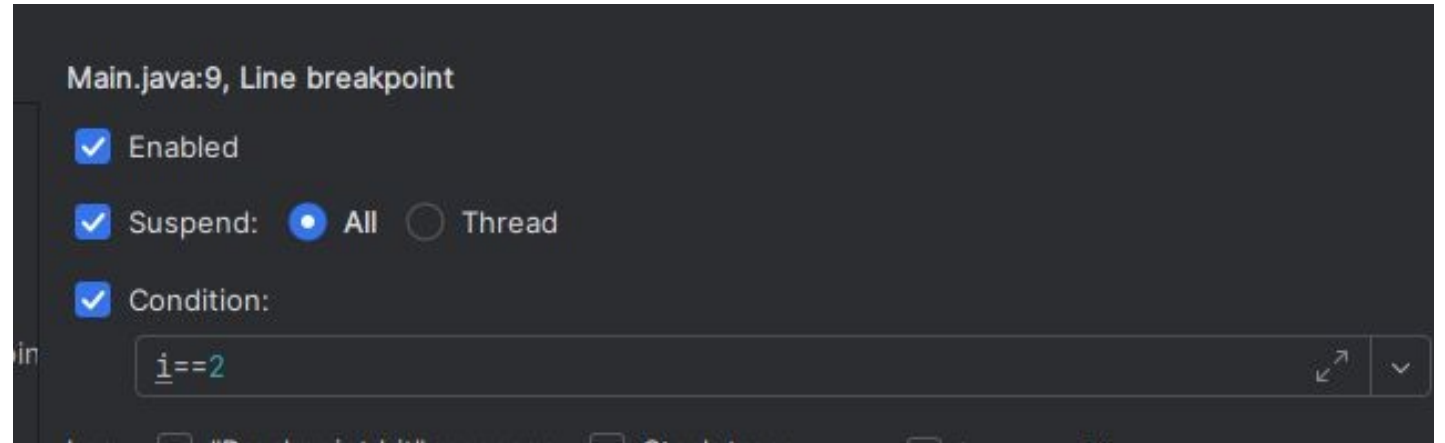
Configuración	Para qué sirve
Condition	Solo se activa si una expresión se cumple (ej. <code>i == 2</code>). Evita parar en cada iteración.
Evaluate and log	En vez de detener el programa, escribe un mensaje en consola.
Remove once hit	El breakpoint se borra tras activarse una vez.
Disable until hitting...	Permanece inactivo hasta que se alcanza otro breakpoint indicado.
Instance filters	Solo se activa para una instancia concreta del objeto.
Class filters	Restringe la activación a una clase concreta.
Pass count	Se activa solo tras alcanzarse un número de veces determinado.
Caller filters	Solo si el código fue invocado desde un método concreto.



Configuraciones avanzadas — Condition

Condition: detenerse solo cuando `i == 2`

Marca la casilla Condition sobre el breakpoint y escribe `i == 2`. El programa solo se detiene en la iteración donde `i` vale exactamente 2, ignorando el resto.

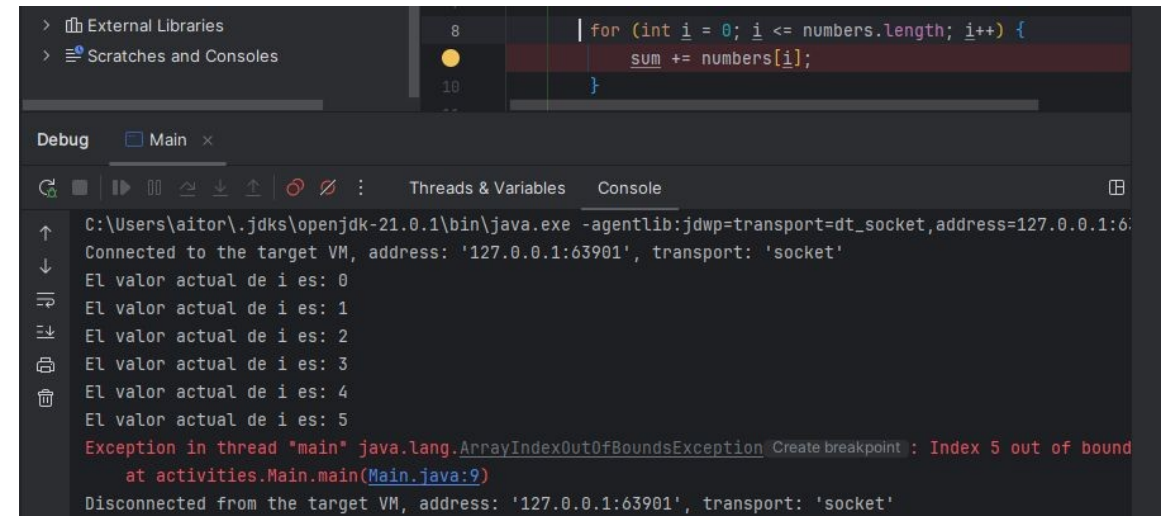
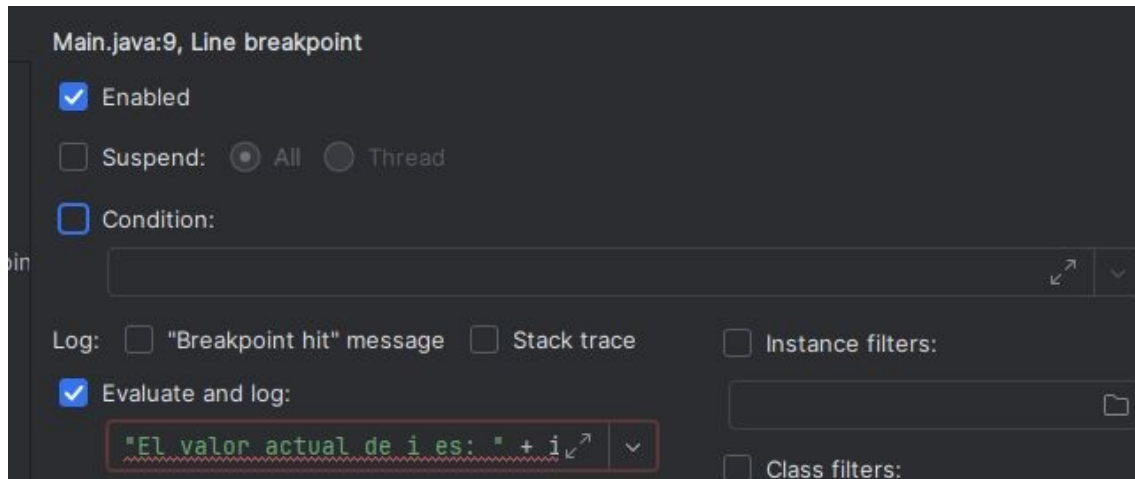


💡 Las opciones Remove once hit, Pass count y Caller filters funcionan de forma similar: cada una añade una condición diferente para que el breakpoint solo se active en circunstancias concretas.

Configuraciones avanzadas — Evaluate and log

Evaluate and log: registrar sin detener

Activa Evaluate and log con el mensaje "El valor actual de i es: " + i y desmarca Suspend. El programa no se detiene: la consola muestra el valor de i en cada vuelta del bucle.

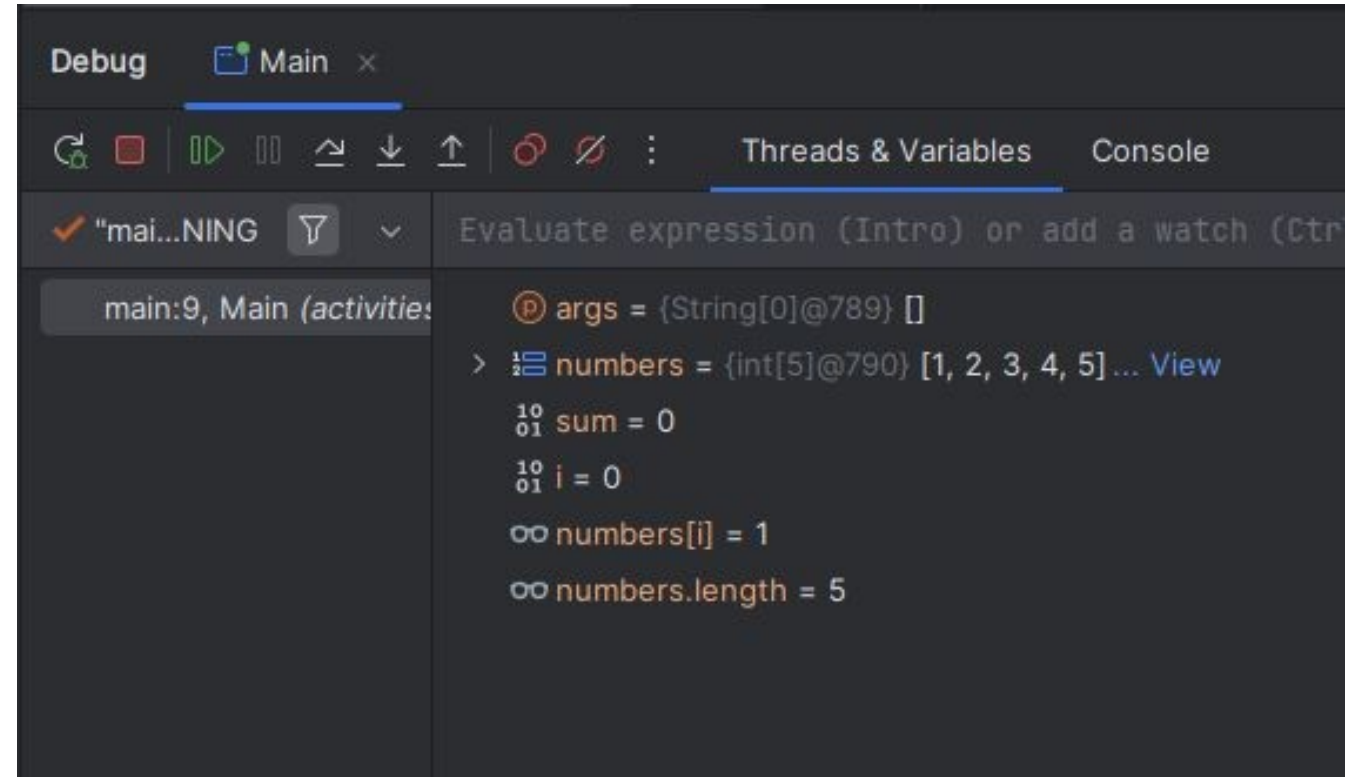


La consola muestra el valor de i en cada iteración hasta que el programa lanza la excepción al salirse del array. El programa nunca se detuvo: solo registró lo que pasó.

Inspeccionar el programa en marcha

Pestaña Variables

- Muestra el valor de todas las variables en el instante de la parada
- Se actualiza automáticamente en cada paso
- Ejemplo: i (índice), numbers[i] (valor), sum (acumulado)

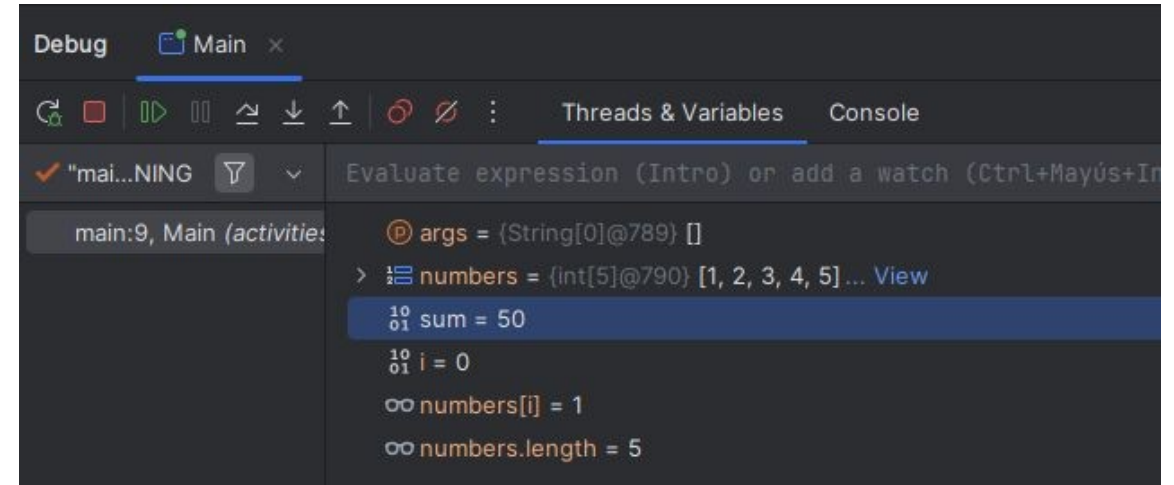


Puedes expandir objetos y arrays directamente en la pestaña Variables para ver sus elementos internos sin necesidad de añadir prints al código.

Modificar y vigilar variables en caliente

Set Value y Watches

- Set Value: clic derecho → Set Value → nuevo valor
- Cambia la variable sin recompilar; el programa sigue con el nuevo valor
- Watches: + New Watch → expresión como `sum + numbers[i]`
- Se recalcula automáticamente en cada parada



El depurador no es solo de lectura. Puedes cambiar variables para probar escenarios distintos sin recompilar, y ver de inmediato cómo afecta ese cambio al resultado.

Avanzar paso a paso

Una vez detenido en un breakpoint, estos son los controles para recorrer el programa:

F8

Step Over

Avanza a la siguiente línea sin entrar en los métodos que se llamen

F7

Step Into

Entra dentro del método llamado en esa línea para depurarlo también

 **F8**

Step Out

Sale del método actual y vuelve al punto desde donde se le llamó

F9

Resume Program

Reanuda la ejecución hasta el siguiente breakpoint (o hasta el final)

Alt+F8

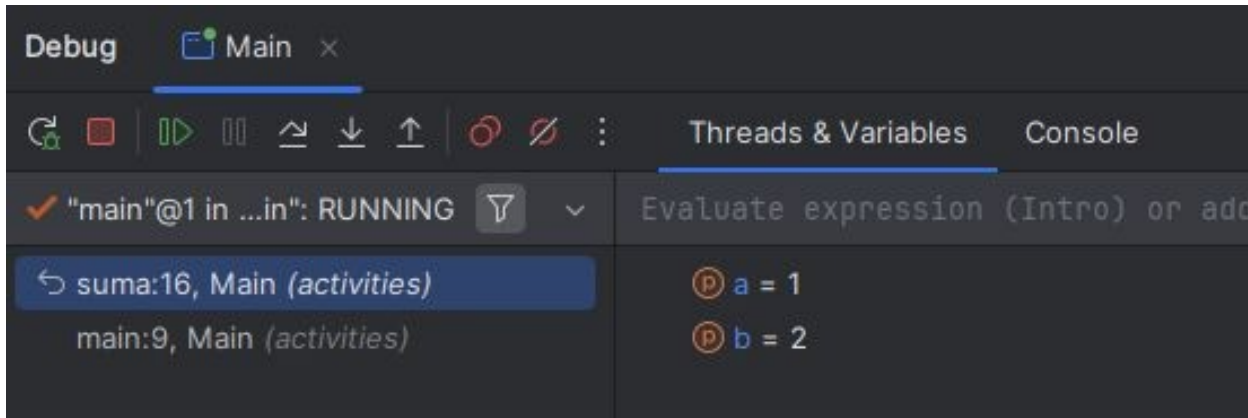
Evaluate Expression

Evalúa cualquier expresión en el contexto actual o reasigna una variable

La pila de llamadas (Call Stack)

La pestaña Frames de la ventana Debug muestra la pila de llamadas: la secuencia de métodos que se han ido llamando unos a otros hasta llegar al punto donde está detenido el programa.

Pestaña Frames — IntelliJ



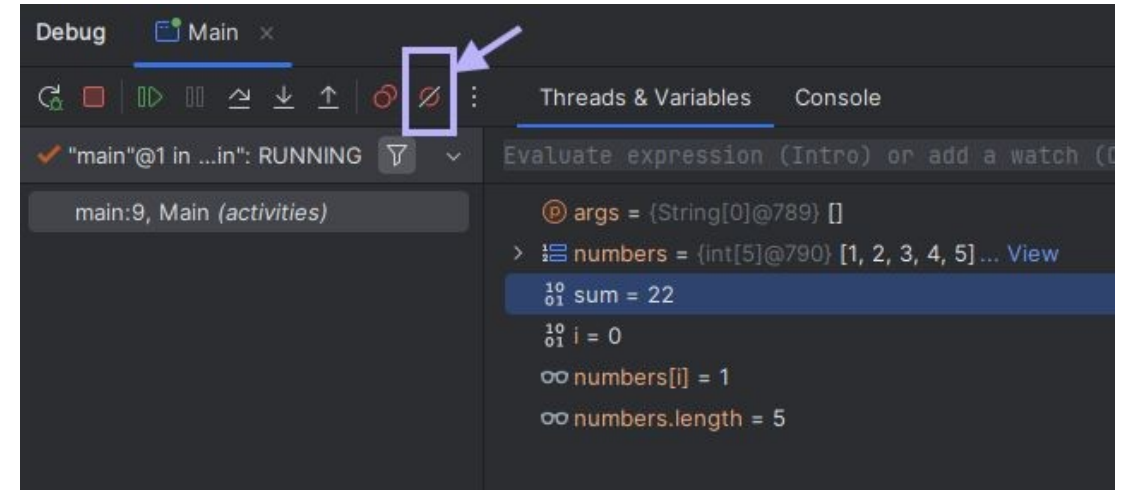
Cómo usarla

- La entrada más alta es el método donde estás parado ahora mismo
- Debajo aparece quien lo llamó, y así sucesivamente
- Haz clic en cualquier línea de la pila para ir directamente a ese punto del código
- Especialmente útil cuando un mismo método se llama desde varios sitios y necesitas saber por cuál has llegado esta vez

Mute Breakpoints — desactivar todos a la vez

Si tienes varios breakpoints colocados pero quieres ejecutar el programa sin que ninguno te interrumpa —sin necesidad de borrarlos uno a uno—, el botón Mute Breakpoints de la ventana Debug los desactiva todos temporalmente.

- Se encuentra en la barra de herramientas de la ventana Debug
- Al pulsarlo, todos los breakpoints se vuelven grises (inactivos) pero no se borran
- Al volver a pulsarlo, se reactivan exactamente como estaban
- Útil cuando quieres ejecutar el programa de principio a fin sin paradas intermedias



💡 Alternativa: desde Run → View Breakpoints (Ctrl+Shift+F8) puedes activar y desactivar breakpoints individuales sin eliminarlos, usando la casilla de verificación de cada uno.